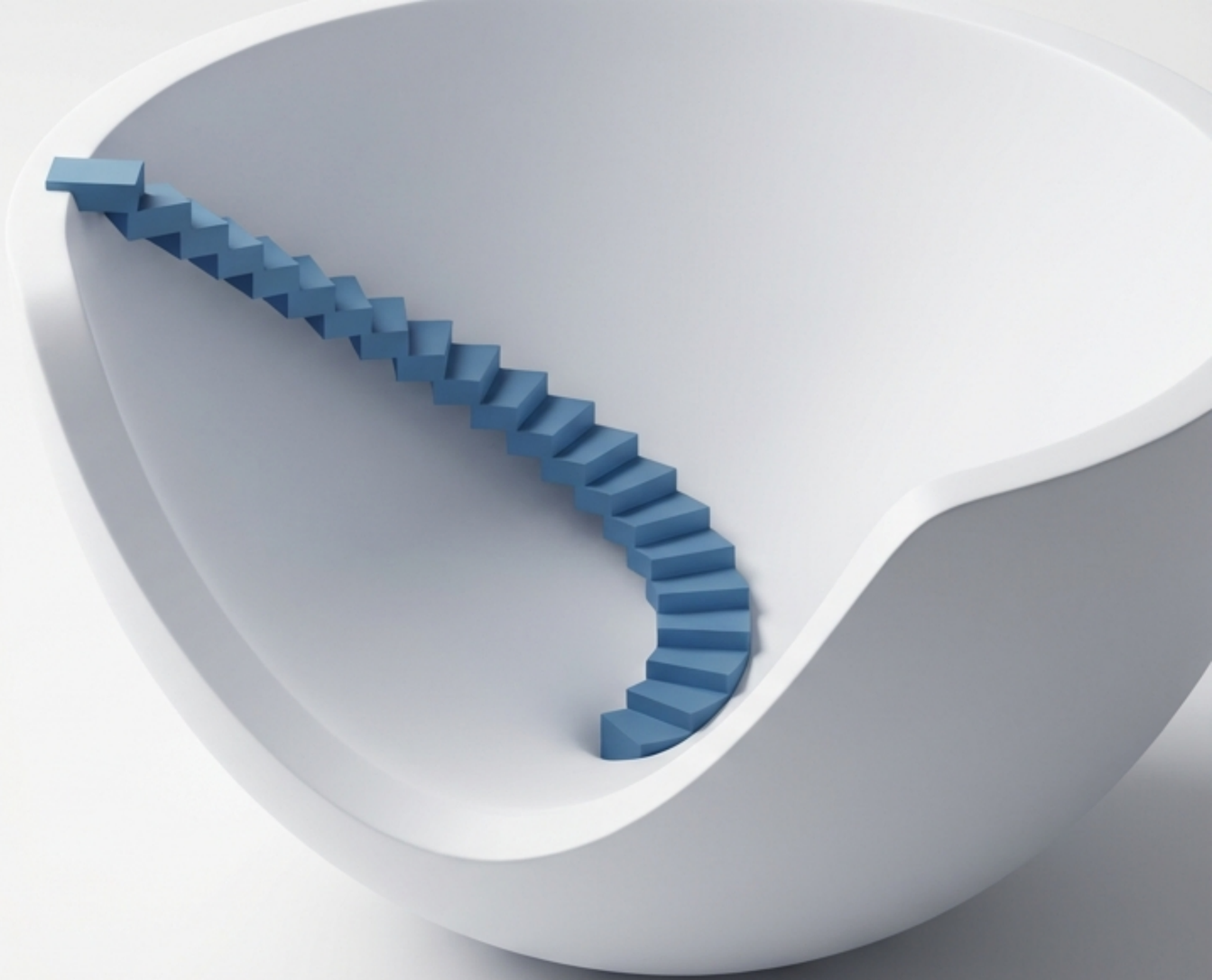
Ridge Regression via Gradient Descent

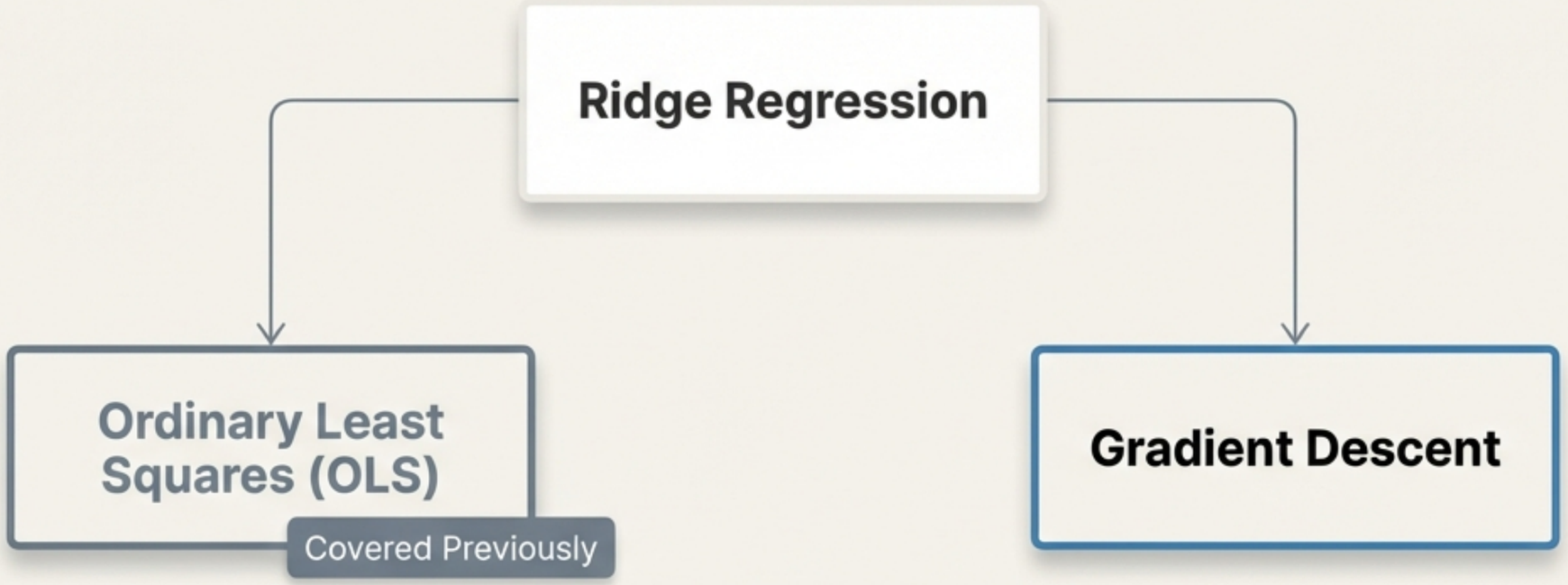
Building the algorithmic engine from scratch to achieve deep mathematical and practical mastery.



Two Paths to Regularisation

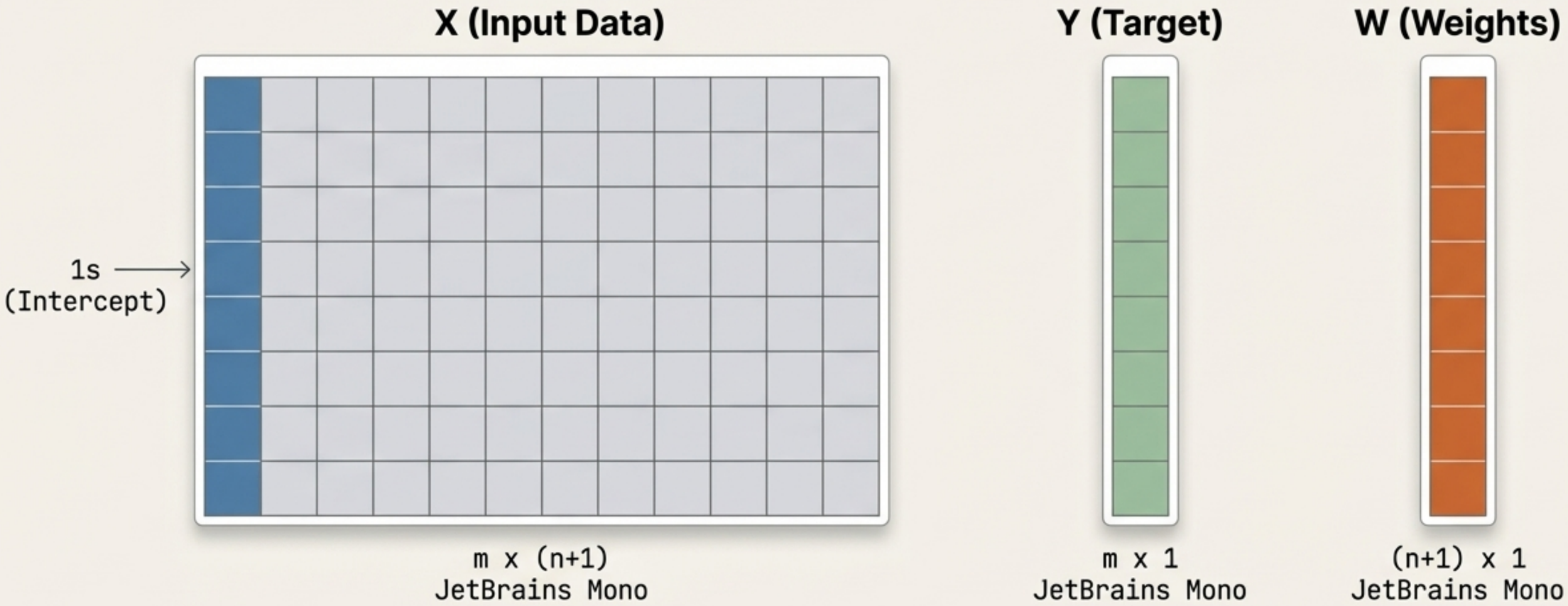
The Goal: Minimise the Ridge loss function iteratively rather than solving it analytically.

Why Gradient Descent? While OLS provides a closed-form solution, Gradient Descent scales efficiently for large datasets and forms the foundation for deep learning optimisation.



Anatomy of the Data Matrices

To formulate the mathematics correctly, we represent our dataset, targets, and parameters in matrix form. The X matrix includes a leading column of 1s to account for the bias term (W_0).



Defining the Vectorised Loss Function

We express the standard linear regression error and the L2 penalty entirely through matrix operations. This vector form allows for efficient calculation and differentiation.

$$L = (XW - Y)^T(XW - Y) + \lambda W^T W$$

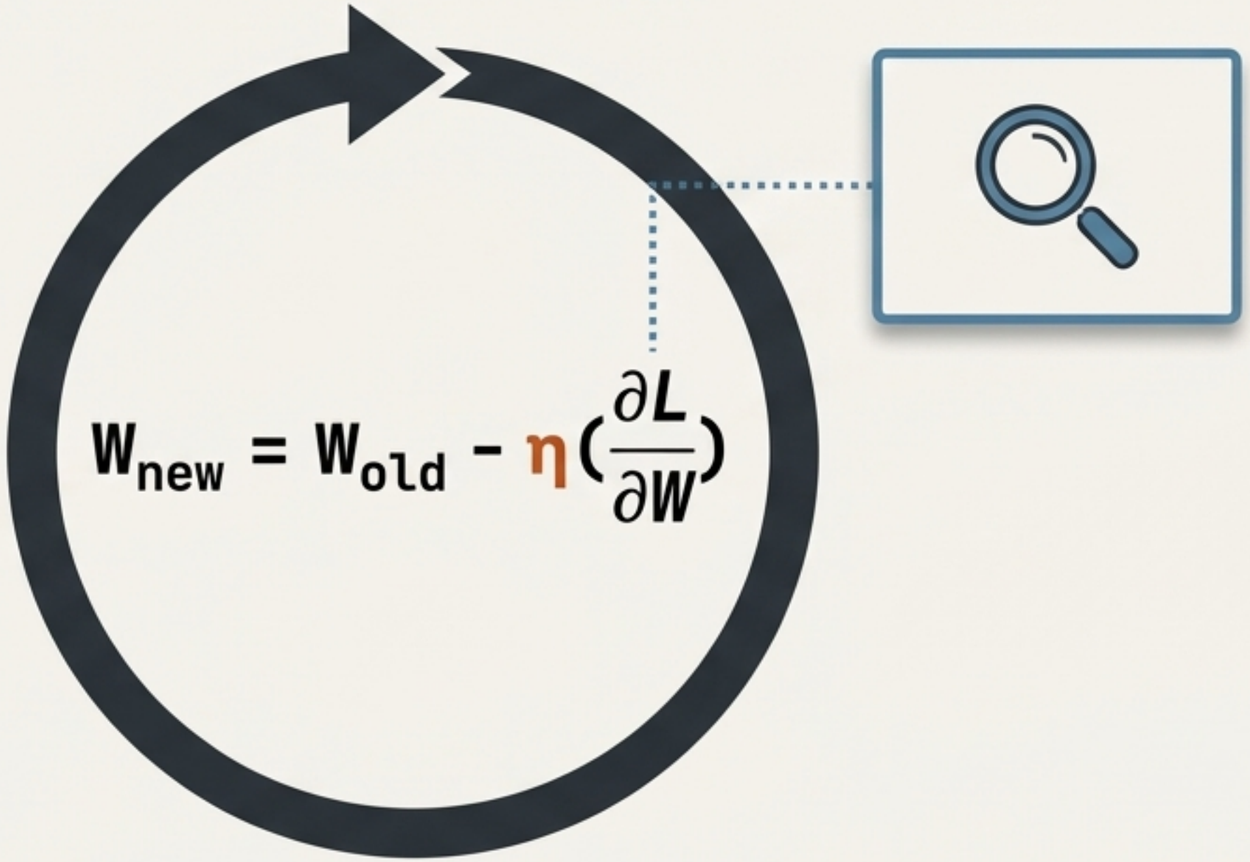
Standard Linear Regression Error
(Residual Sum of Squares)

L2 Regularisation Penalty
(The Ridge)

The Gradient Descent Engine

η (Learning Rate): Controls the step size.

The Challenge: To update the weight matrix **W**, we must calculate the derivative of the entire loss function matrix with respect to **W**. This is our immediate mathematical target.



Deriving the Gradient: Expansion

We expand the transposed terms. For mathematical convenience during differentiation, we multiply the entire loss function by 1/2. This scales the loss but does not change the location of the minimum, ensuring cleaner derivatives later.

1. Original:

$$L = (XW - Y)^T(XW - Y) + \lambda W^T W$$

2. Expanded:

$$L = W^T X^T XW - Y^T XW - W^T X^T Y + Y^T Y + \lambda W^T W$$

3. Convenience Adjustment:

$$L = \frac{1}{2}\left[W^T X^T XW - Y^T XW - W^T X^T TY + Y^T Y + \lambda W^T W\right]$$

Matrix Calculus: A Common Pitfall

When differentiating with respect to the matrix **W**, it is a common mistake to misalign the transposed terms. The correct derivative of the cross-terms yields **$-X^TY$**. Correctly applying matrix calculus rules is essential to ensure the gradient points in the true direction of descent.

Incorrect Alignment

Correct Differentiation

The Final Gradient Equation

With the differentiation complete and the constants cancelled out, we arrive at the precise calculation required for our update loop.

X^TXW: Current model predictions scaled by features.
X^TY: Actual target mappings.
λW: The Ridge penalty scaling the weights.

$$\frac{\partial L}{\partial W} = X^T X W - X^T Y + \lambda W$$

Implementation Path 1: SGDRegressor

Scikit-learn provides out-of-the-box functionality. The SGDRegressor applies Stochastic Gradient Descent. By setting the penalty='l2', we invoke Ridge Regression. In our test environment, this yields a baseline R^2 score of 0.44.

```
model = SGDRegressor(penalty='l2',
    max_iter=500, eta0=0.1, alpha=0.001)

model.fit(X_train, y_train)
```

R^2 Score: 0.44

Implementation Path 2: Ridge with Solvers

Alternatively, Scikit-learn's dedicated Ridge class can bypass its default closed-form math by explicitly setting the solver. Using solver='sag' (Stochastic Average Gradient) directs the model to use gradient descent, slightly improving our baseline to 0.46.

```
model = Ridge(alpha=0.1, max_iter=500,
    solver='sag')

model.fit(X_train, y_train)
```

R^2 Score: 0.46

Building from Scratch: Initialisation

To truly understand the mechanics, we build a custom class (`MeraRidge`). We initialise our parameters (**learning rate**, **epochs**, **alpha**) and construct our `W` matrix (`self.theta`) starting with ones. Crucially, we insert a column of `1s` into `X` to handle the intercept cleanly.

```
class MeraRidge:
    def __init__(self, epochs, learning_rate, alpha):
        self.epochs = epochs
        self.lr = learning_rate
        self.alpha = alpha
        self.theta = None

    def fit(self, X_train, y_train):
        self.theta = np.ones(X_train.shape[1] + 1)
        X_train = np.insert(X_train, 0, 1, axis=1)
```

Math to Code: The Update Loop

The theoretical derivation maps 1:1 with NumPy operations. The power of vectorised mathematics is that complex loops are replaced by highly optimised matrix multiplications.

THE MATHS

$$\frac{\partial L}{\partial W} = X^T XW - X^T Y + \lambda W$$

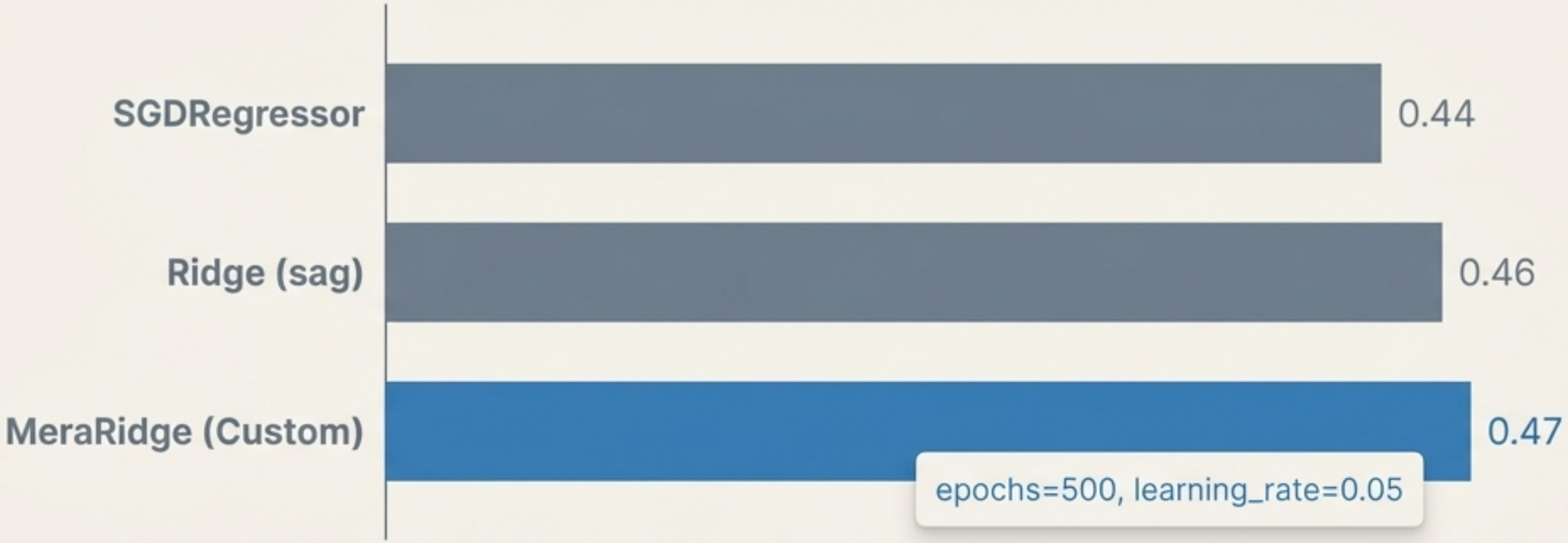
THE CODE

```
derivative = X.T @ (X @ W) - X.T @ Y + self.alpha * W

W = W - self.lr * derivative
```

Performance Evaluation

Running our custom batch gradient descent algorithm matches and even marginally outperforms the default library implementations in this specific test. The standard libraries abstract away learning rate schedules, but our manual control proves the underlying maths is perfectly sound.

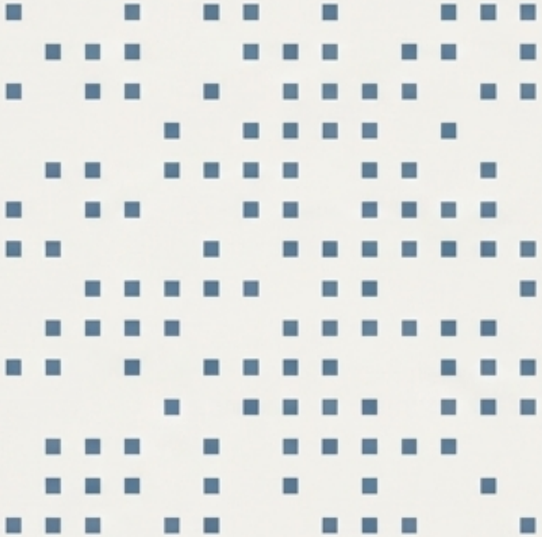


Beyond Batch Optimisation

We have successfully implemented Batch Gradient Descent—processing the entire dataset per epoch. To handle massive datasets efficiently, the exact same mathematical principles can be adapted into Mini-Batch or Stochastic Gradient Descent (SGD) architectures.

Batch

Mini-Batch

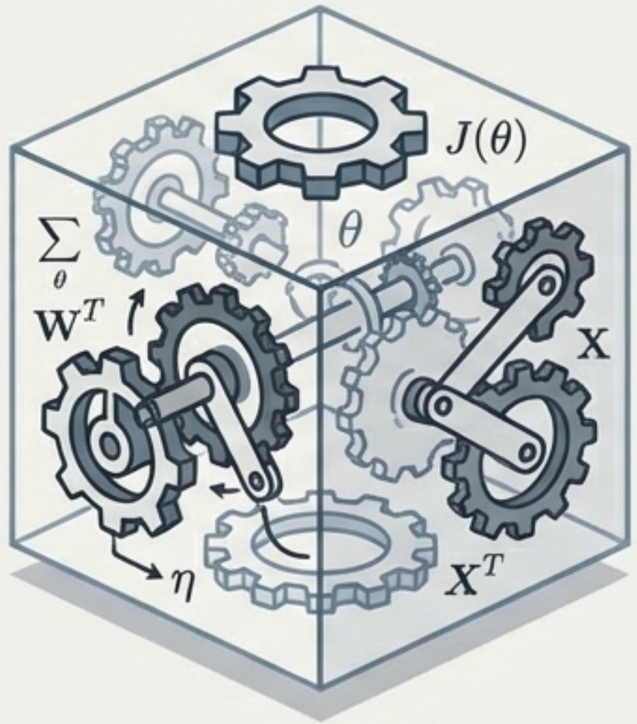


Stochastic

The Value of End-to-End Construction

Many practitioners rely exclusively on pre-built libraries, treating regularisation as a black box. By deconstructing the loss function, deriving the matrix calculus, and writing the update loops from scratch, you transition from simply using tools to truly understanding them.

The Black Box



Mathematical Mastery